Software Engineering

BCSE301L

Module 1

Dr. Naveenkumar Jayakumar
Dept. of Computational Intelligence
PRP-217-4

Software - Definition

Software is defined as the tool with designated order of instructions, performing tasks to provide the desired results after obtaining the requirements of the users. It also consists of procedures, rules, software-related documents and the associated basic data required to run

Software = Computer Programs (Instructions) + Procedures + Rules + Associated documents + Data

Nature of Software - Characteristics

- **Intangibility:** Software cannot be physically touched or perceived by our senses. It exists as code and data, residing within the memory and storage devices of a computer.

- **Complexity:** As software systems grow in size and functionality, they become increasingly complex. This complexity arises from the intricate interplay of numerous components and their interactions.

Nature of Software - Characteristics

- **Conformity:** Software must conform to specific hardware and software platforms. It needs to be compatible with the operating system, programming languages, and other software components it interacts with.

- **Changeability:** Software is highly malleable and can be modified or extended relatively easily. This flexibility allows for continuous improvement and adaptation to changing requirements.

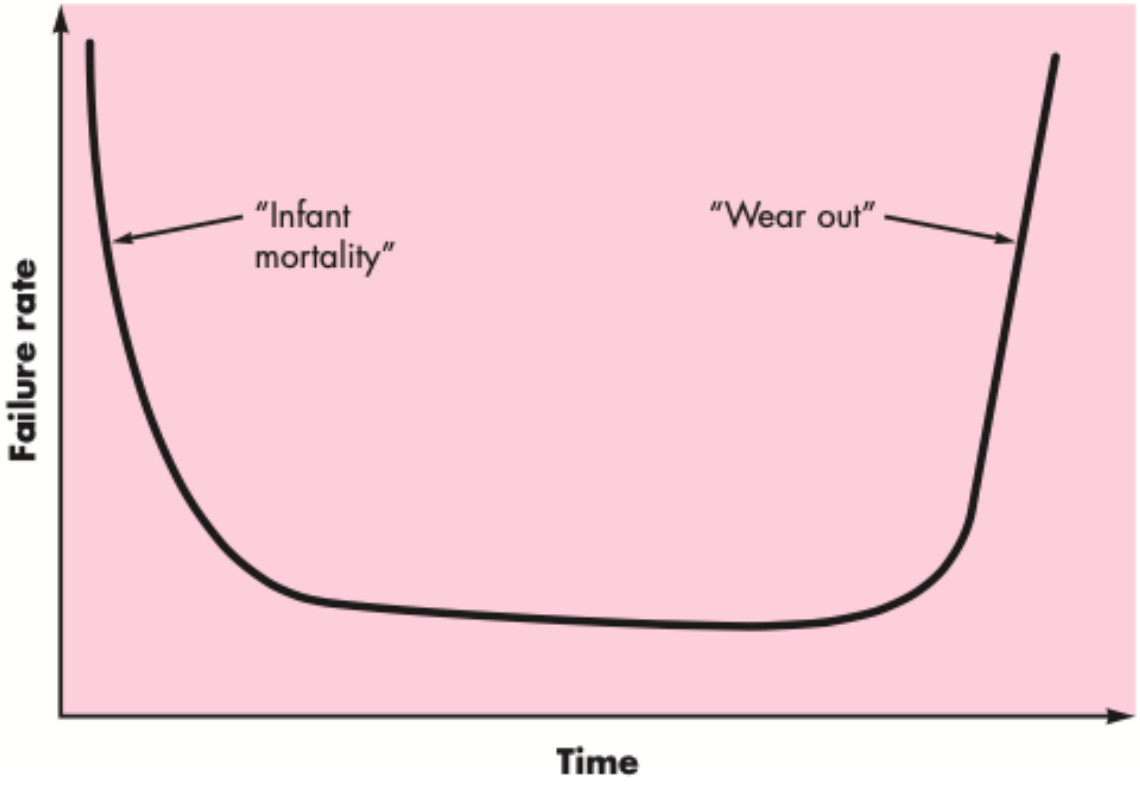
Nature of Software - Characteristics

- **Invisibility:** The internal workings of software are often hidden from view. Users interact with the software's output, but the underlying code and algorithms remain invisible.

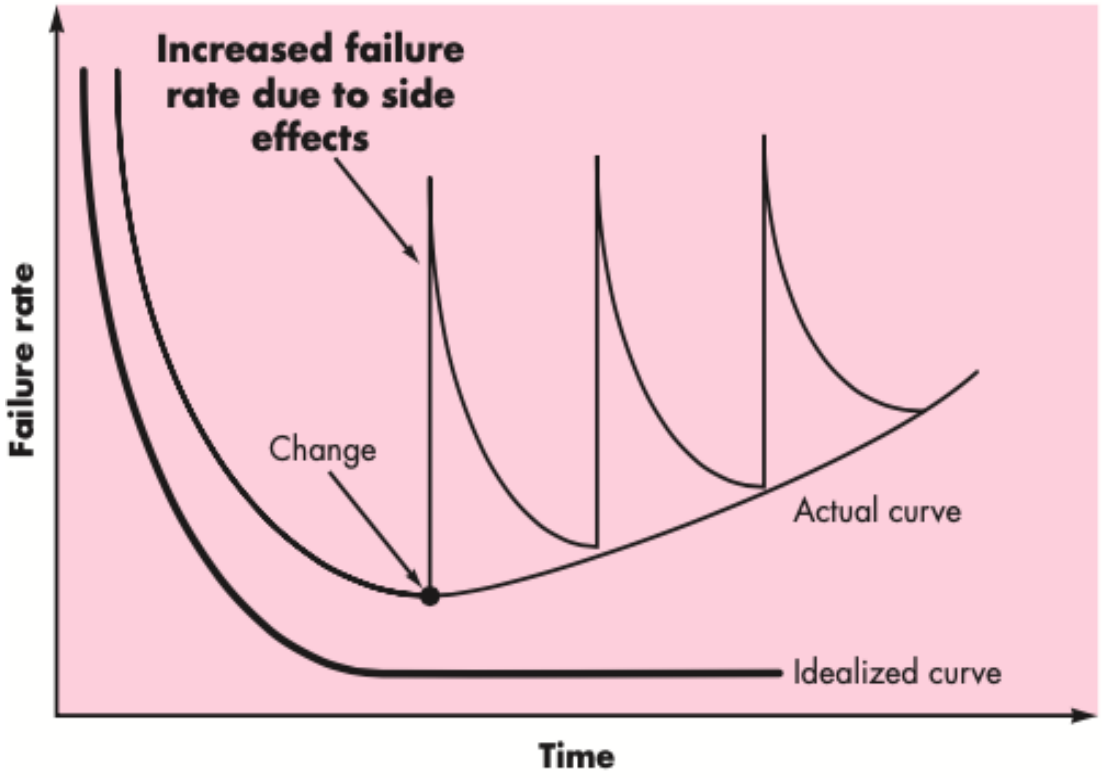
Nature of Software – Comparing to Hardware

- Software is developed or engineered; it is not manufactured
- Software doesn't wear out. But Undiscovered defects will cause high failure rates early in the life of a program.
- Although the industry is moving toward component-based construction, most software continues to be custom built.

Nature of Software – Comparing to Hardware



Failure curves for Hardware



Failure curves for software

Nature of Software – Application Domain

- System Software
- Web Application
- Embedded Software
- Mobile Software
- Engineering/Scientific
- Artificial Intelligence
- Data Science Applications

Nature of Software - Characteristics

- Implications of Software nature
  - **Challenges in Development**: The intangible and complex nature of software makes it challenging to predict and manage development efforts accurately.
  - **Quality Assurance**: Ensuring the quality and reliability of software requires rigorous testing and debugging processes.
  - **Intellectual Property**: Software is often protected by copyright laws to safeguard the rights of its creators.
  - **Maintenance and Evolution**: Software requires ongoing maintenance and updates to address bugs, security vulnerabilities, and evolving user needs.

Software Engineering - Definition

The application of a systematic, disciplined, quantifiable approach to the development, operation and maintenance of software; that is, the application of engineering to software. The study of these approach is Software Engineering

Software Engineering

Software engineering is an *engineering discipline* that is concerned with *all aspects of software production* from the early stages of system specification through to maintaining the system after it has gone into use.

- Engineering Discipline - apply theories, methods, and tools
- All aspects of software production - Technical processes, Planning, budgeting, Managing, manpower, development of tools, methods, and theories

Software Engineering – A layered Approach

Software Engineering – A layered Approach

- Quality Focus
  - Organizational Commitment to Quality: Philosophies like the following must be practiced
    - Total Quality Management
    - Six Sigma
    - Continuous Process Improvement Culture

Software Engineering – A layered Approach

- Software Engineering **Process**:
  - Framework for effective software delivery
  - Management control of software projects
  - Context for applying technical methods
  - Establishment of milestones
  - Quality assurance
  - Change management

Software Engineering – A layered Approach

- Software Engineering **Methods:**
  - Technical **how-to's** for building software
  - Communication
  - Requirements analysis
  - Design modeling
  - Program construction
  - Testing
  - Support
  - Basic principles governing software engineering
  - Modeling activities and descriptive techniques

Software Engineering – A layered Approach

- Software Engineering **Tools:**
  - Automated or semi-automated support for the process and methods
  - Computer-aided software engineering (CASE) for integrated tool support

Software Process

- *A software process is a sequence of **activities**, **actions** and **tasks** that leads to the production of a software product.*
  - **An activity** strives to achieve a broad objective (e.g., communication with stakeholders)
  - **An action** (e.g., architectural design) encompasses a set of tasks that produce a major work product (e.g., an architectural design model).
  - **A task** focuses on a small, but well-defined objective (e.g., conducting a unit test) that produces a tangible outcome.

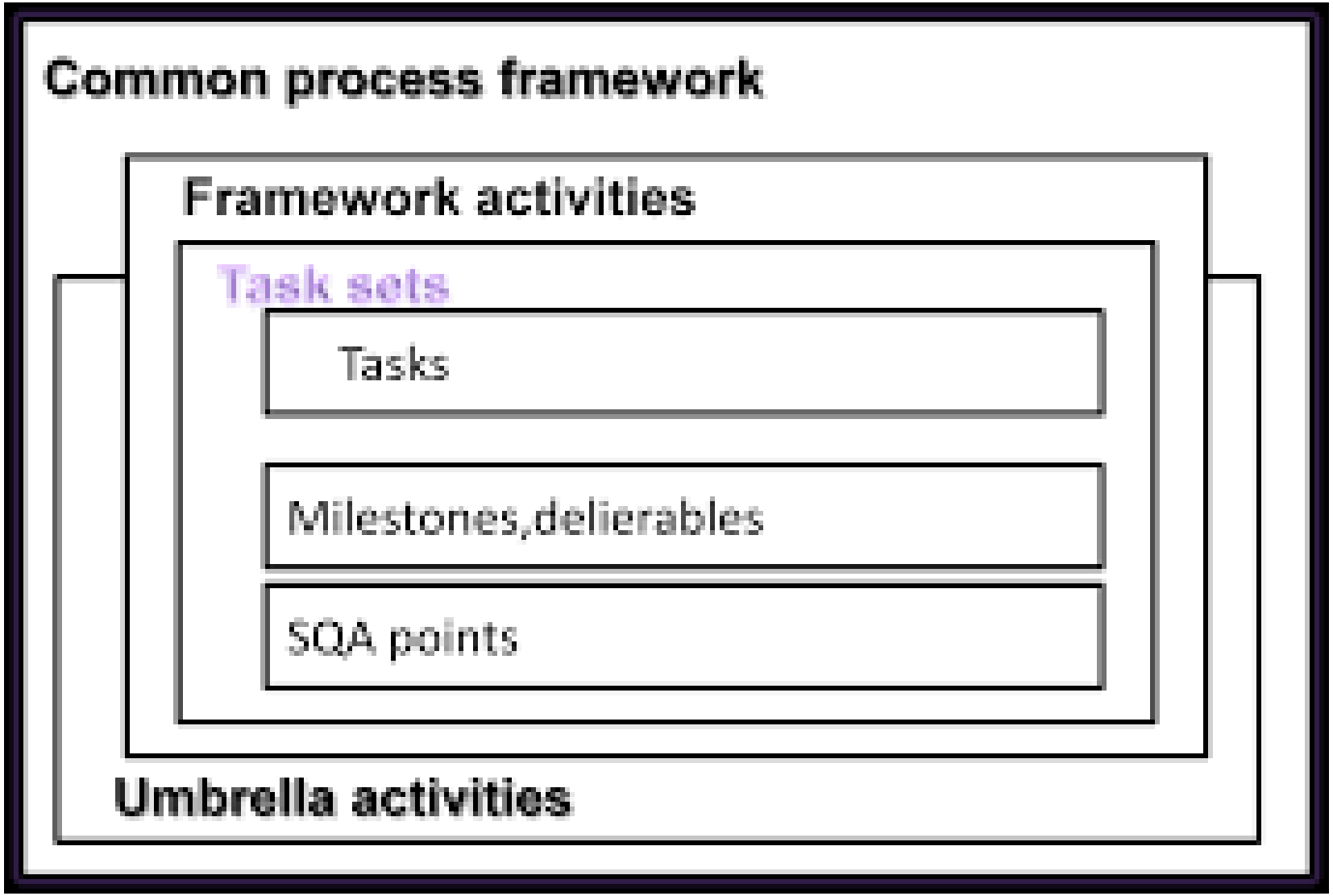
Software Process – Fundamental Activities

- Software specification, where **customers and engineers** define the software that is to be produced and the **constraints on its operation**.
- Software development, where the software is **designed and programmed**.
- Software validation, where the software is **checked to ensure** that it is what the customer requires.
- Software evolution, where the software is **modified to reflect changing** customer and market requirements.

Software Process – Framework

- A process framework is like a basic blueprint for building software.
- It outlines the main steps or activities needed for any software project, no matter how big or small. These main steps are called framework activities.
- In addition to these main steps, there are also **umbrella activities** that apply to the entire software development process.
- These umbrella activities help to manage and control the project as a whole.

Software Process – Framework



Software Process – Generic Framework Activities

Communication	Planning	Modelling	Construction	Deployment
• Understanding Stakeholder Requirements • Gather Requirements	• Create a Software Project Plan • Define Technical tasks, risks, resources and work products	• Create models to understand software requirements and design	• Code Generation • Testing	• Deliver software to the customer • Evaluate customer feedback

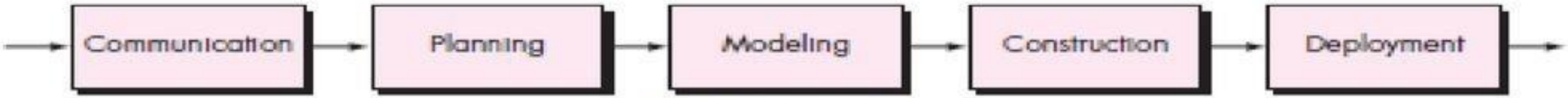
Software Process – Umbrella Activities

Project Tracking and Control:	Assess project progress against the plan
	Take corrective actions to maintain the schedule
Risk Management:	Identify and assess project risks
Quality Assurance:	Define and conduct quality assurance activities
Technical Reviews:	Review software work products to find and fix errors

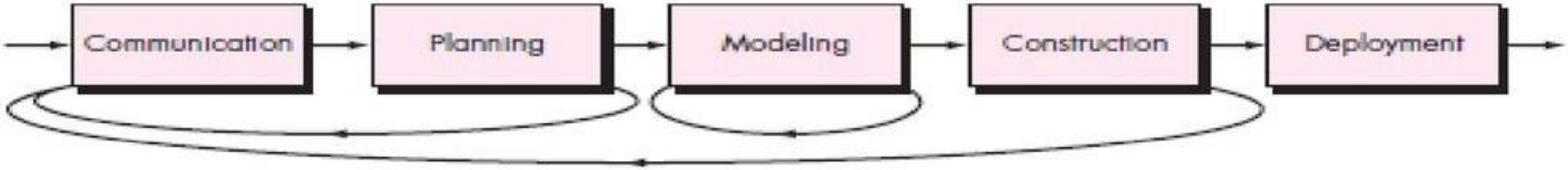
Software Process – Umbrella Activities

Measurement:	Define and collect project metrics
	Use metrics to improve the process
Configuration Management:	Manage changes to the software
Reusability Management:	Identify and reuse software components
Work Product Production:	Create software work products (models, documents, etc.)

Software Process – Activities Flow Pattern

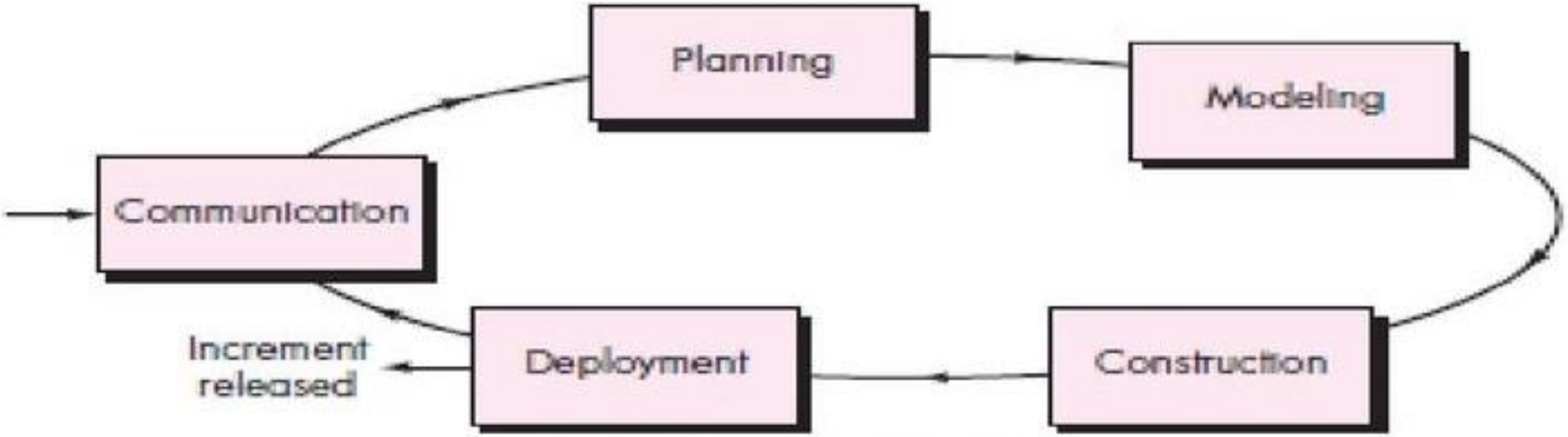


(a) Linear process flow



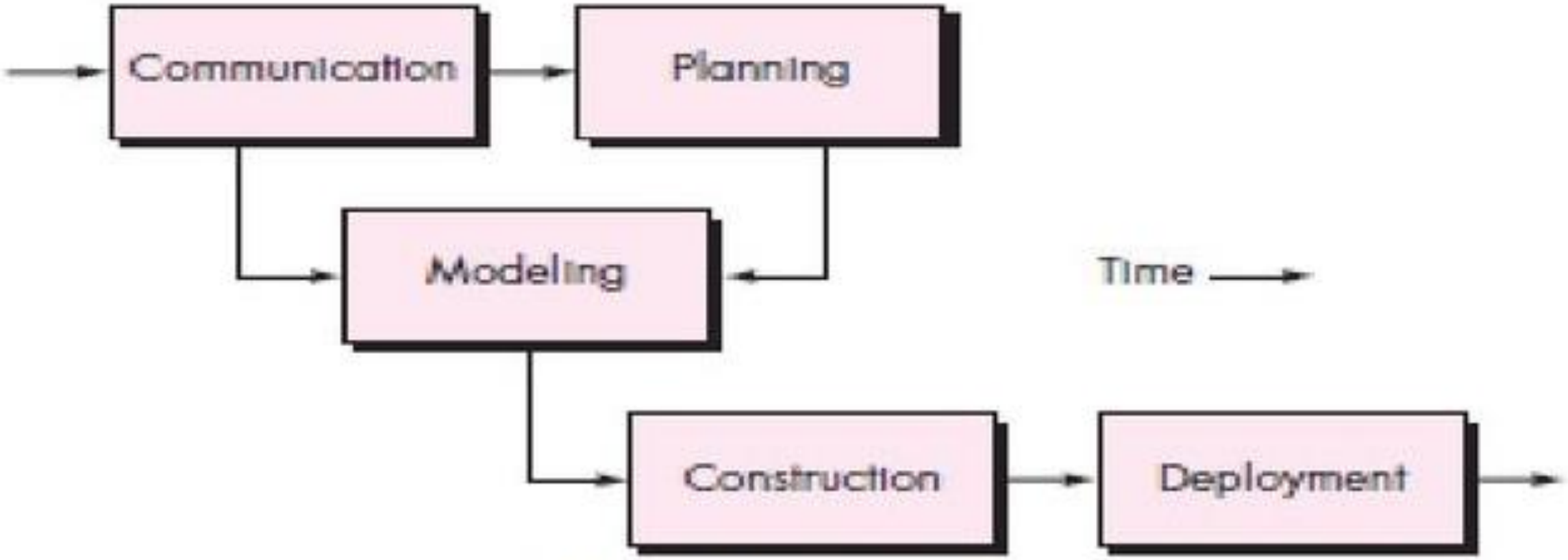
(b) Iterative process flow

Software Process – Activities Flow Pattern



(c) Evolutionary process flow

Software Process – Activities Flow Pattern



(d) Parallel process flow

The 4Ps Constituting Software Engineering

People

- Project stakeholders

Product

- The software product plus associated documents.

Project

- The activities carried out to produce the product.

Process

- Framework within which the team carries out the activities necessary to build the product.

The 4Ps Constituting Software Engineering

People	Product	Project	Process
• Business Management People • Project Management People • Development Team members • Customers • Vendors • End Users	• Project documentation -Documents produced during software definition and development. • Code - Source and object. • Text documents - Plans, cases, and results. • Customer documents - Documents explaining how to use and operate product. • Productivity measurements - Analyse project productivity.	• Planning - Plan, monitor, and control the software project. • Requirements analysis - Define what to build. • Design - Describe how to build the software. • Implementation - Program the software. • Testing - Validate that software meets the requirements. • Maintenance - Resolve problems; adapt software to meet new requirements.	• Framework Activities • Umbrella Activities

The Process Models

Details the Software Development Process

Provides a Visual representation of Development Process

Helps Efficient development of software

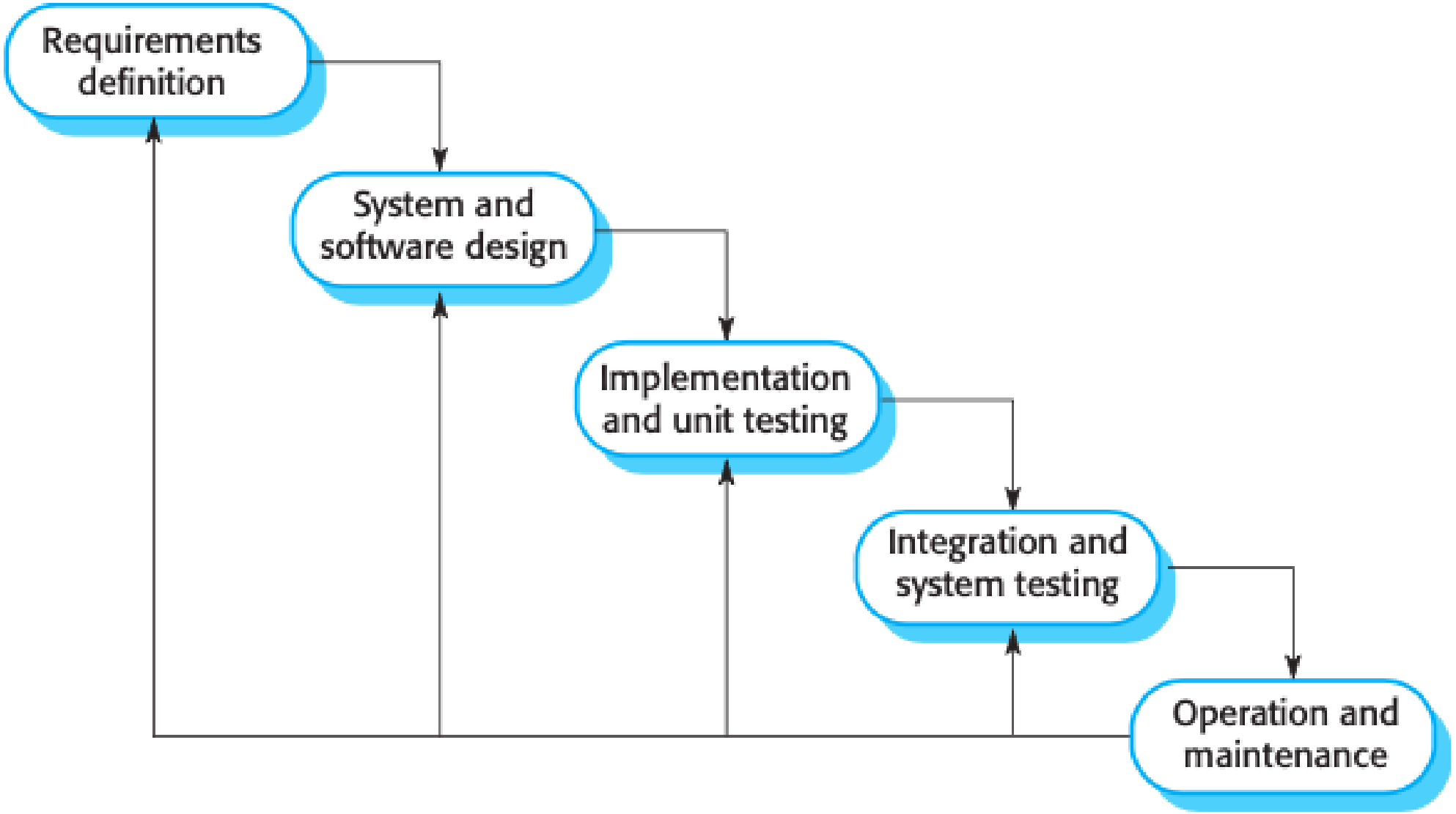
a framework that outlines the stages, activities, and tasks

structured approach to software development

projects are completed on time, within budget

Satisfies the required quality standards.

The Process Models – Waterfall Model



The Process Models – Waterfall Model

- **Characteristics**
  - **Linear and Sequential**: Each phase is completed before moving on to the next one.
  - **Predictive**: Requirements are gathered and defined before the project begins.
  - **Phased**: The project is divided into distinct phases.

The Process Models – Waterfall Model

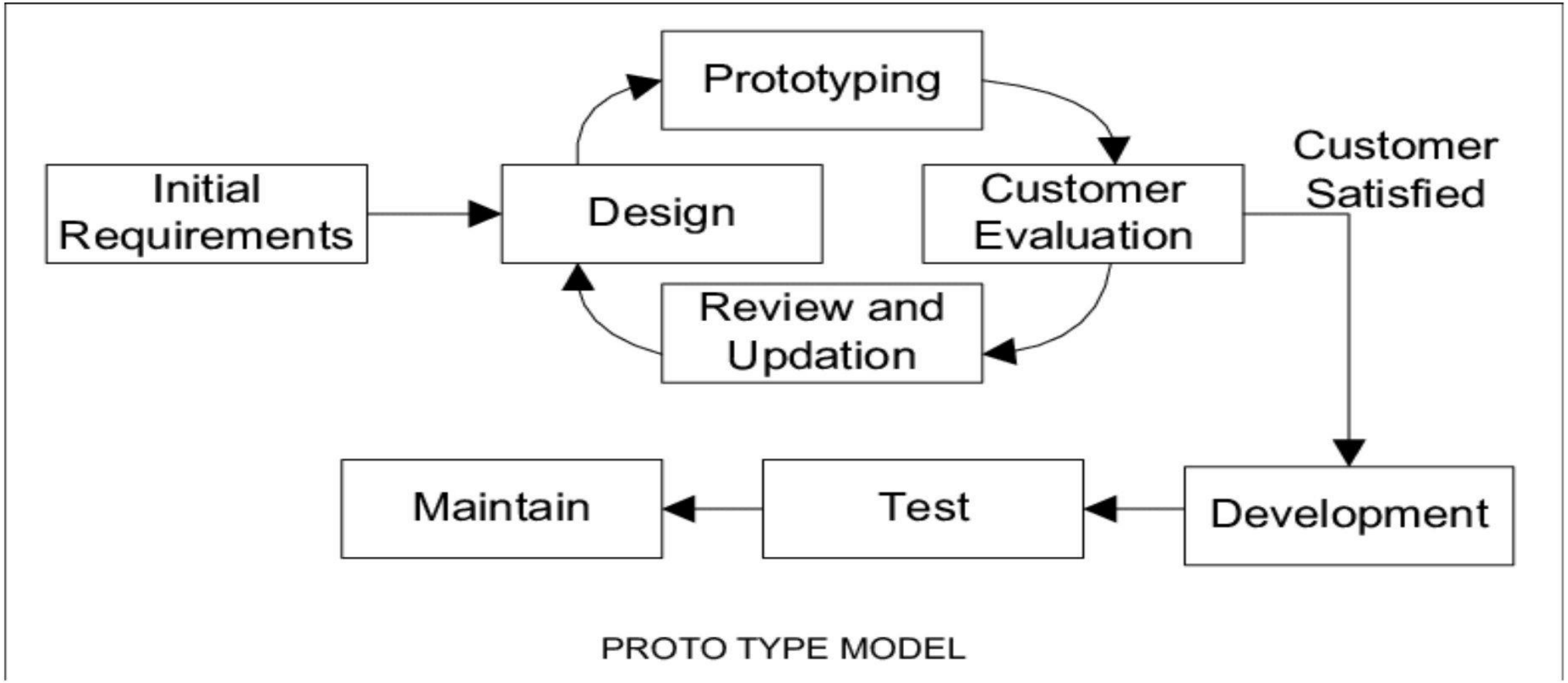
- **Phases of the Waterfall Model**
  - **Requirements Gathering**: Collect and document requirements from stakeholders.
  - **Analysis**: Analyze the requirements to identify the project's scope and constraints.
  - **Design**: Create a detailed design of the software, including architecture, components, and interfaces.
  - **Implementation:** Write the code for the software.
  - **Testing**: Test the software to identify and fix defects.
  - **Deployment**: Deploy the software to production.
  - **Maintenance**: Maintain and update the software after deployment.

The Process Models – Waterfall Model

- **Advantages**
  - **Easy to Manage**: The linear and sequential approach makes it easy to manage and understand.
  - **Well-defined Phases**: Each phase has a clear set of deliverables and expectations.
  - **Predictable Timelines**: The predictive nature of the model allows for accurate timeline estimation.

- **Disadvantages**
  - **Inflexible**: Changes to requirements can be difficult and costly to implement.
  - **High Risk**: If requirements are not properly understood or defined, the project may fail.
  - **Long Development Time:** The sequential approach can lead to long development times.

The Process Models – Prototyping Model



The Process Models – Prototyping Model

- **Characteristics**
  - **Iterative:** The prototyping process is iterative, with each iteration refining the prototype.
  - **Flexible:** The prototyping model allows for changes in requirements and design.
  - **Risk Reduction:** Prototyping helps to reduce the risk of project failure by identifying and addressing potential issues early.

The Process Models – Prototyping Model

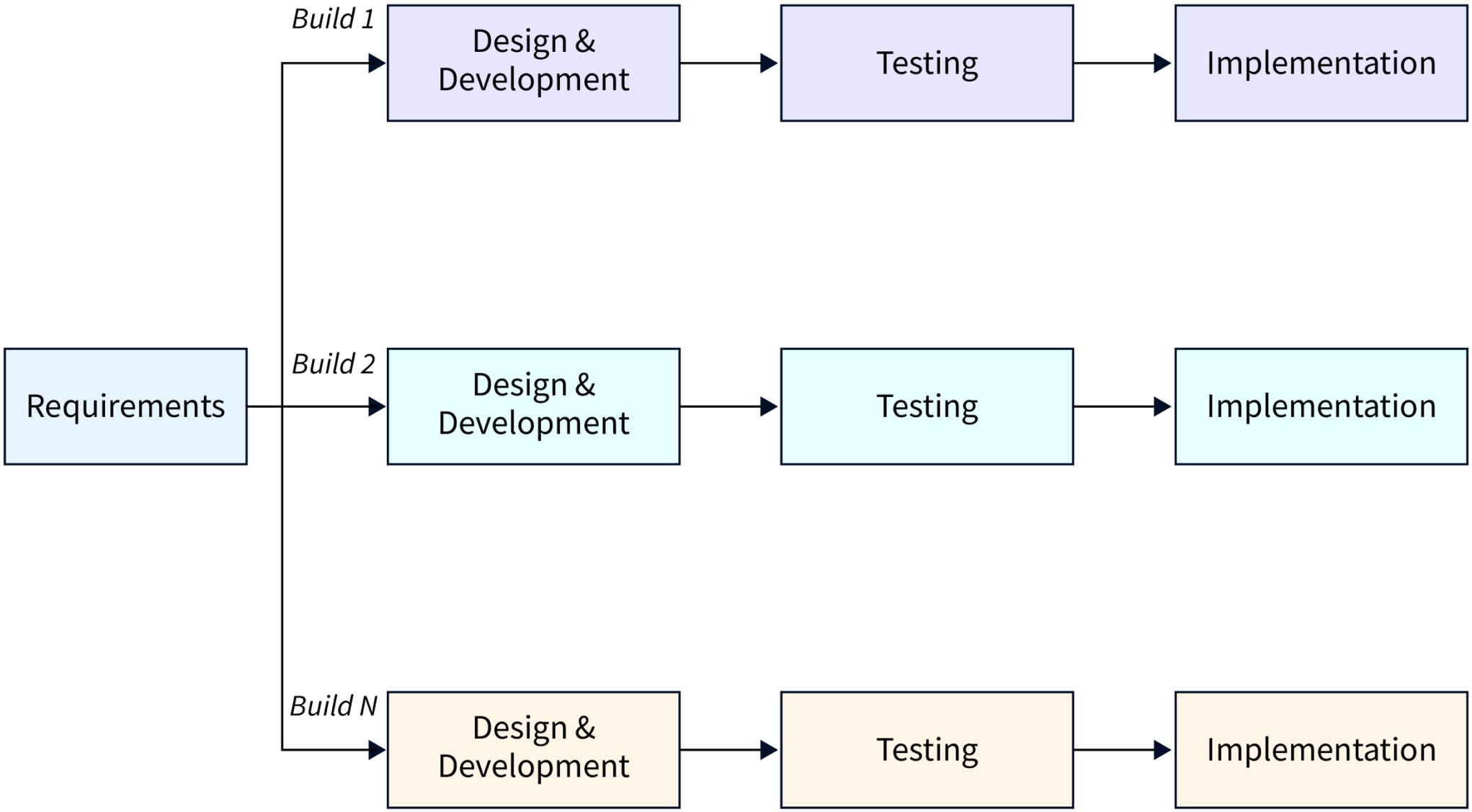
- **Phases of the Prototyping Model**
  - **Requirements Gathering:** Identify the basic requirements of the software application.
  - **Quick Plan:** Create a rough plan for the prototype.
  - **Prototype Development:** Build a working prototype of the software application.
  - **User Feedback:** Gather feedback from users on the prototype.
  - **Refine Prototype:** Refine the prototype based on user feedback.
  - **Repeat:** Repeat the process until the prototype meets the requirements.

The Process Models – Prototyping Model

- **Advantages**
  - **Improved Requirements**: Prototyping helps to identify and refine requirements.
  - **Reduced Risk:** Prototyping reduces the risk of project failure.
  - **Increased User Satisfaction**: Prototyping ensures that the final product meets user requirements.

- **Disadvantages**
  - **Time-consuming:** Prototyping can be time-consuming, especially if multiple iterations are required.
  - **Expensive:** Prototyping can be expensive, especially if a large team is involved.
  - **May Not Be Suitable for Large Projects:** Prototyping may not be suitable for large projects with complex requirements.

The Process Models – Incremental Model



The Process Models – Incremental Model

- **Characteristics**
  - **Evolutionary:** The incremental model involves multiple evolution, with each iteration adding new features or functionality.
  - **Incremental:** The software application is developed in increments, with each increment building on the previous one.
  - **Flexible:** The incremental model allows for changes in requirements and design.

The Process Models – Incremental Model

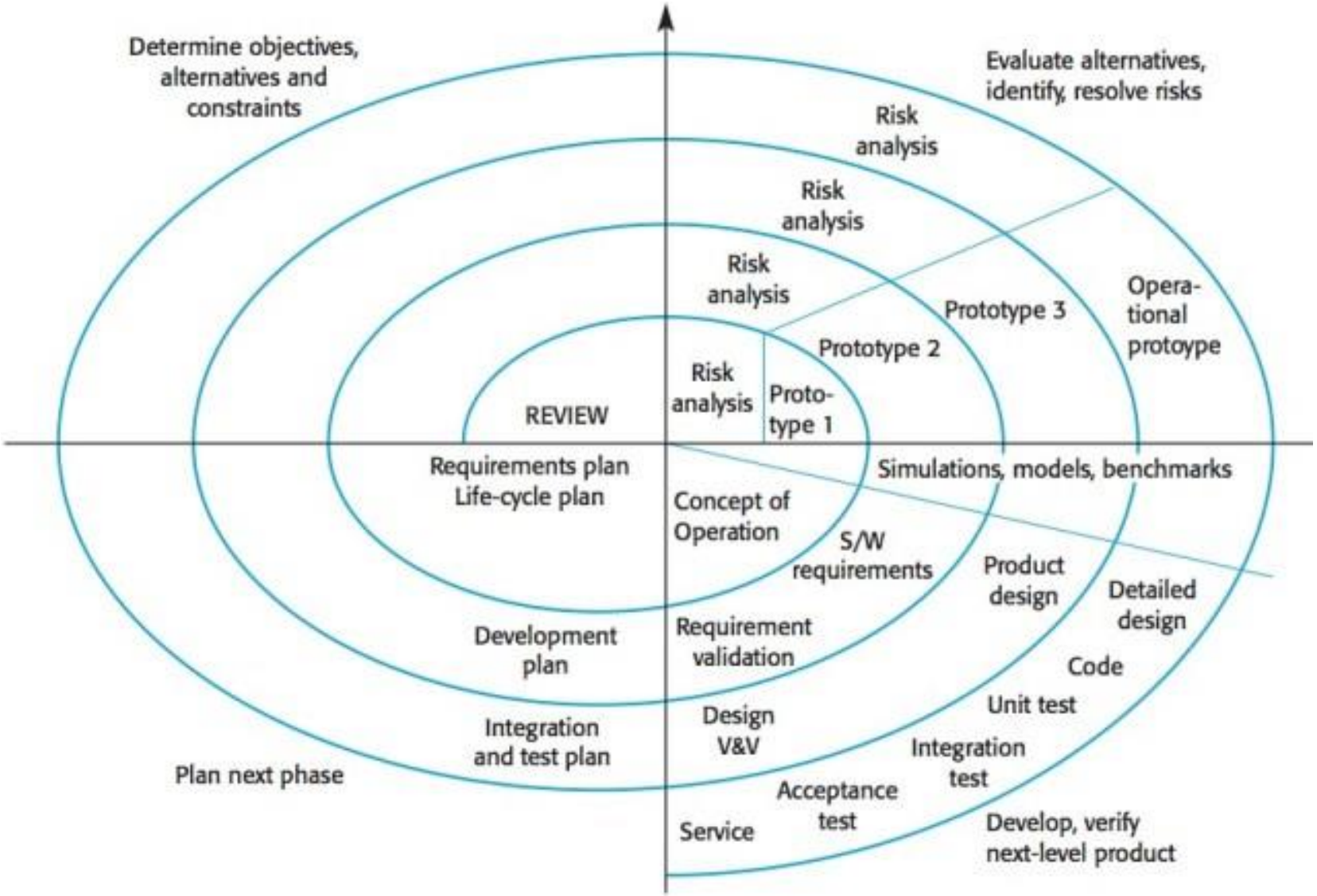
- **Phases of the Incremental Model**
  - **Requirements Gathering:** Identify the overall requirements of the software application.
  - **System Design**: Create a high-level design of the software application.
  - **Incremental Development**: Develop the software application in increments, with each increment adding new features or functionality.
  - **Testing**: Test each increment to ensure that it meets the requirements.
  - **Integration**: Integrate each increment into the overall software application.
  - **Repeat**: Repeat the process until the software application is complete.

The Process Models – Incremental Model

- **Advantages**
  - **Early Delivery**: The incremental model allows for early delivery of a working software application.
  - **Reduced Risk**: The incremental model reduces the risk of project failure by breaking the project into smaller, more manageable increments.
  - **Improved Quality**: The incremental model allows for testing and integration of each increment, improving the overall quality of the software application.

- **Disadvantages**
  - **Higher Cost**: The incremental model can be more expensive than other models, as it requires multiple iterations and testing.
  - **Requires Good Planning**: The incremental model requires good planning and coordination to ensure that each increment is properly integrated into the overall software application.
  - **May Not Be Suitable for Large Projects**: The incremental model may not be suitable for large projects with complex requirements.

The Process Models – Spiral Model



The Process Models – Spiral Model

- **Characteristics**
  - **Iterative & Evolutionary:** The Spiral Model involves multiple iterations, with each iteration refining the software application.
  - **Incremental**: The software application is developed in increments, with each increment building on the previous one.
  - **Risk-driven**: The Spiral Model emphasizes risk management and mitigation.
  - **Flexible**: The Spiral Model allows for changes in requirements and design.

The Process Models – Spiral Model

- **Phases of the Spiral Model**
  - **Planning:** Identify the objectives, constraints, and risks of the project.
  - **Risk Analysis:** Analyze the risks associated with the project and develop strategies to mitigate them.
  - **Engineering:** Develop the software application, including design, implementation, and testing.
  - **Evaluation:** Evaluate the software application and identify areas for improvement.
  - **Repeat:** Repeat the process until the software application meets the requirements.

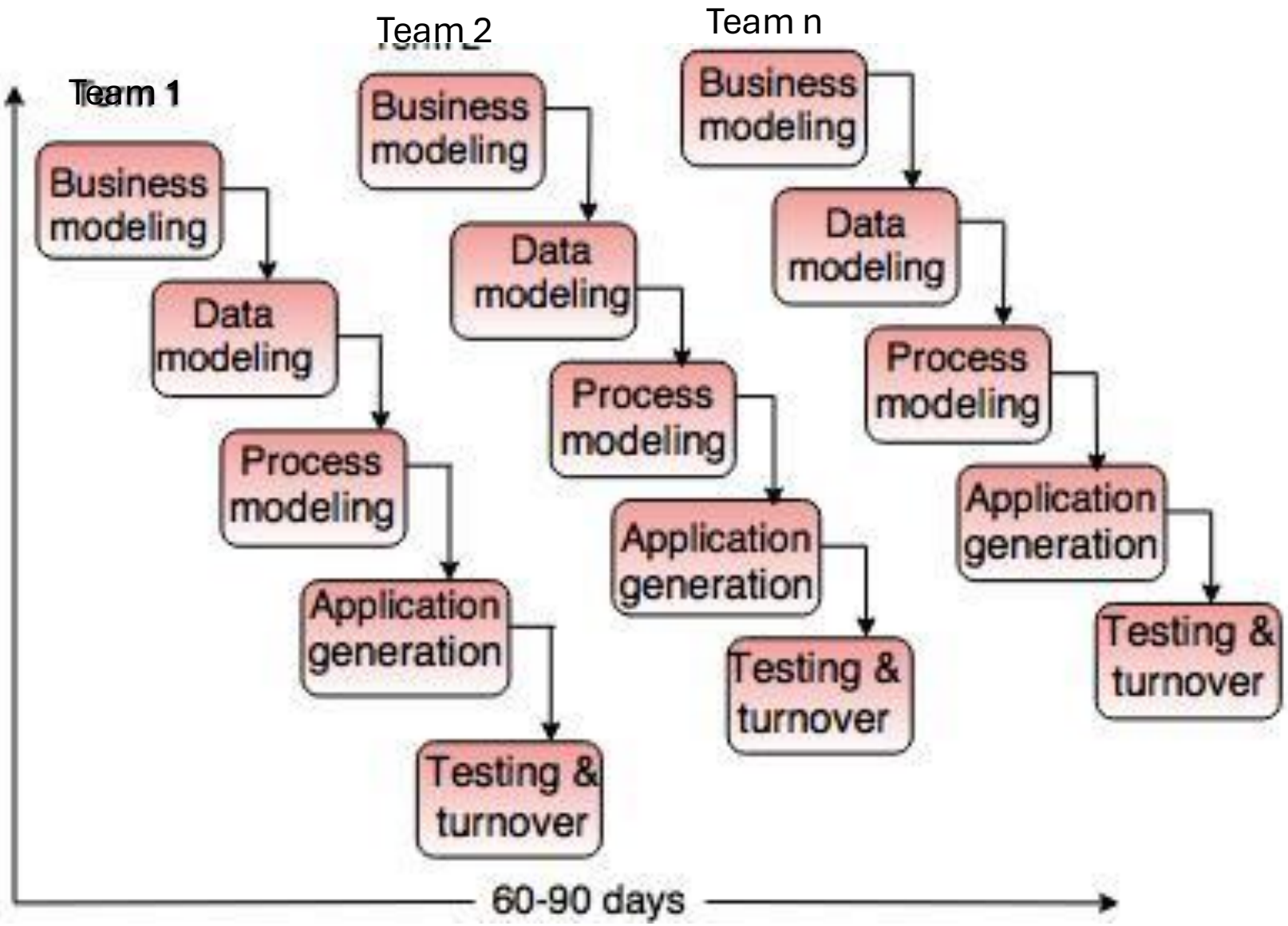
The Process Models – Spiral Model

- **Advantages**
  - **Risk Management:** The Spiral Model emphasizes risk management and mitigation.
  - **Flexibility:** The Spiral Model allows for changes in requirements and design.
  - **Improved Quality:** The iterative and incremental approach of the Spiral Model results in improved quality.
  - **Reduced Risk:** The Spiral Model reduces the risk of project failure by identifying and mitigating risks early.

The Process Models – Spiral Model

- **Disadvantages**
  - **Complexity:** The Spiral Model can be complex and difficult to manage.
  - **High Cost:** The Spiral Model can be expensive, as it requires multiple iterations and risk analysis.
  - **Requires Skilled Resources:** The Spiral Model requires skilled resources, including risk management and software development expertise.

The Process Models – RAD Model



The Process Models – RAD Model

- **Characteristics**
  - **Rapid Development:** The RAD Model emphasizes rapid development and delivery of a software application.
  - **Iterative:** The RAD Model involves multiple iterations, with each iteration refining the software application.
  - **Incremental:** The software application is developed in increments, with each increment building on the previous one.
  - **User Involvement:** The RAD Model emphasizes user involvement and feedback throughout the development process.

The Process Models – RAD Model

- Phases of the RAD Model
  - **Business Modeling:**
  - Business processes and information flow are analyzed to identify the core business requirements.
  - Stakeholders and developers collaborate to create a high-level overview of what the system will do.
  - **Data Modeling:**
  - Information derived from business modeling is refined into data objects (entities).
  - Relationships between data objects are defined, establishing a clear data structure.

The Process Models – RAD Model

- Phases of the RAD Model
  - **Process Modeling:**
    - Data objects and their relationships are used to define the processes (functions) that manipulate data.
    - These processes are refined iteratively for greater clarity and alignment with user requirements.
  - **Application Generation:**
    - Actual coding and development occur.
    - Tools like CASE (Computer-Aided Software Engineering) are often used to automate code generation, speeding up development.
  - **Testing & Turnover:**
    - Components are tested independently and integrated into the system.
    - Bugs are identified and resolved rapidly.
    - A working product is delivered for review and feedback.

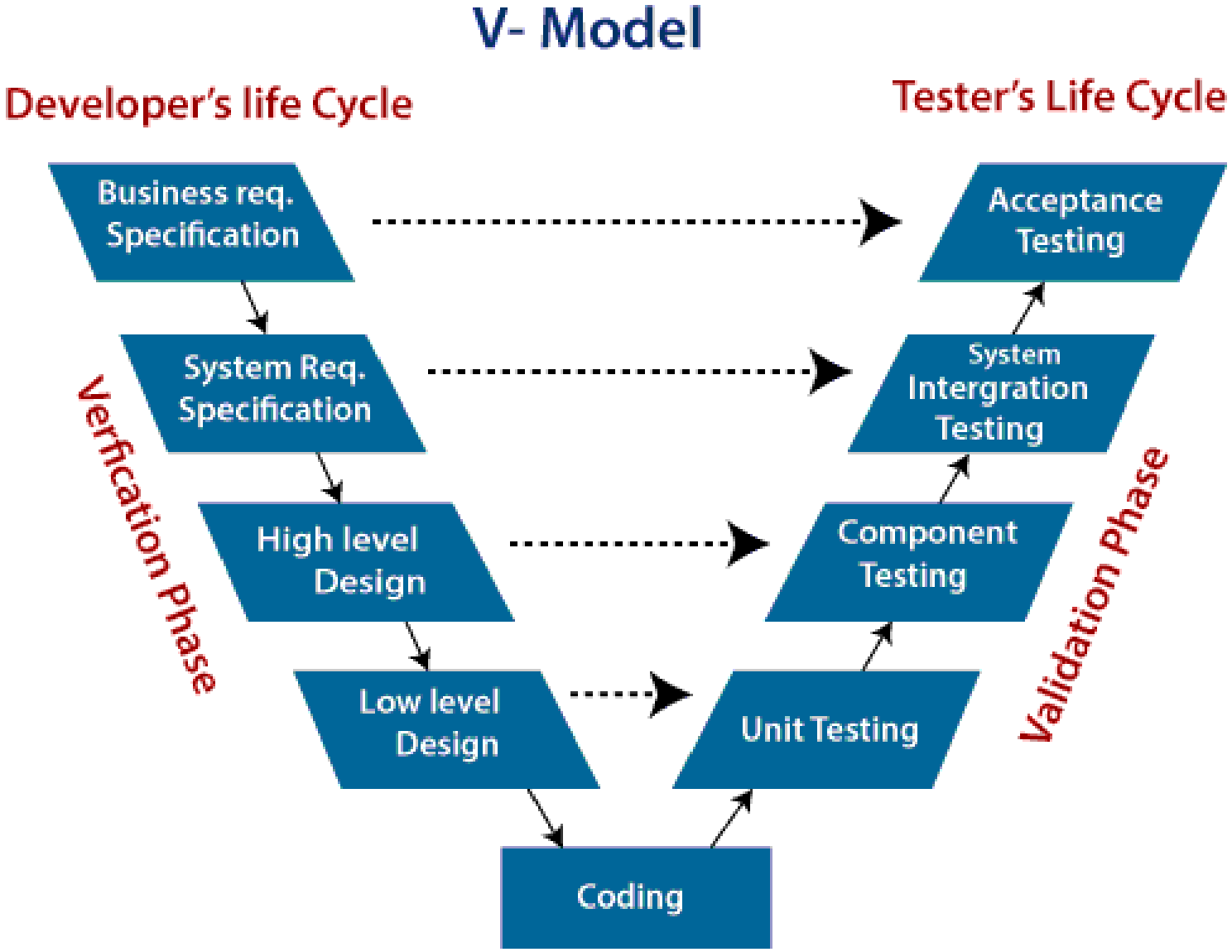
The Process Models – RAD Model

- **Advantages**
  - **Rapid Development:** The RAD Model enables rapid development and delivery of a software application.
  - **Improved User Satisfaction:** The RAD Model emphasizes user involvement and feedback, resulting in improved user satisfaction.
  - **Flexibility:** The RAD Model allows for changes in requirements and design.
  - **Reduced Risk:** The RAD Model reduces the risk of project failure by involving users throughout the development process.

The Process Models – RAD Model

- **Disadvantages**
- **High Cost:** The RAD Model can be expensive, as it requires rapid development and deployment.
- **Requires Skilled Resources:** The RAD Model requires skilled resources, including developers, designers, and testers.
- **May Not Be Suitable for Large Projects:** The RAD Model may not be suitable for large projects with complex requirements.

The Process Models – V Model



The Process Models – V Model

- **Characteristics**
  - **Sequential & Parallel Phases:** The V-Model consists of sequential phases, with each phase building on the previous one.
  - **Testing Emphasis:** The V-Model places a strong emphasis on testing, with testing activities integrated throughout the development process.
  - **Verification and Validation:** The V-Model includes verification and validation activities to ensure that the software meets the requirements.

The Process Models – V Model - Phases

- **Business requirement analysis**: This is the first step where product requirements understood from the customer's side. This phase contains detailed communication to understand customer's expectations and exact requirements.
- **System Design:** In this stage system engineers analyze and interpret the business of the proposed system by studying the user requirements document.
- **Architecture Design**: The baseline in selecting the architecture is that it should understand all which typically consists of the list of modules, brief functionality of each module, their interface relationships, dependencies, database tables, architecture diagrams, technology detail, etc. The integration testing model is carried out in a particular phase.

The Process Models – V Model - Phases

- **Module Design:** In the module design phase, the system breaks down into small modules. The detailed design of the modules is specified, which is known as Low-Level Design
- **Coding Phase:** After designing, the coding phase is started. Based on the requirements, a suitable programming language is decided. There are some guidelines and standards for coding. Before checking in the repository, the final build is optimized for better performance, and the code goes through many code reviews to check the performance.

The Process Models – V Model - Phases

- There are the various phases of Validation Phase of V-model:
- **Unit Testing:** In the V-Model, Unit Test Plans (UTPs) are developed during the module design phase. These UTPs are executed to eliminate errors at code level or unit level. A unit is the smallest entity which can independently exist, e.g., a program module. Unit testing verifies that the smallest entity can function correctly when isolated from the rest of the codes/ units.
- **Integration Testing:** Integration Test Plans are developed during the Architectural Design Phase. These tests verify that groups created and tested independently can coexist and communicate among themselves.

The Process Models – V Model - Phases

- **System Testing:** System Tests Plans are developed during System Design Phase. Unlike Unit and Integration Test Plans, System Tests Plans are composed by the client's business team. System Test ensures that expectations from an application developer are met.
- **Acceptance Testing:** Acceptance testing is related to the business requirement analysis part. It includes testing the software product in user atmosphere. Acceptance tests reveal the compatibility problems with the different systems, which is available within the user atmosphere. It conjointly discovers the non-functional problems like load and performance defects within the real user atmosphere.

The Process Models – V Model

- **Advantage (Pros) of V-Model:**
  - Easy to Understand.
  - Testing Methods like planning, test designing happens well before coding.
  - This saves a lot of time. Hence a higher chance of success over the waterfall model.
  - Avoids the downward flow of the defects.
  - Works well for small plans where requirements are easily understood.

The Process Models – V Model

- **Disadvantage (Cons) of V-Model:**
  - Very rigid and least flexible.
  - Not a good for a complex project.
  - Software is developed during the implementation stage, so no early prototypes of the software are produced.
  - If any changes happen in the midway, then the test documents along with the required documents, has to be updated.

The Process Models – Agility

- **Agility** in Software Engineering refers to the *ability to adapt and respond quickly to changes* throughout the software development lifecycle.
- It emphasizes *flexibility, collaboration, and continuous improvement* to deliver high-quality software efficiently.

The Process Models – Agile Process Model

- **Key Characteristics of Agile Methods**
- **Iterative Development:**
  - System is developed in small increments.
  - Each increment is delivered and evaluated by users.
  - Feedback from users is used to refine future increments.
- **Minimal Documentation:**
  - Focus on code and working software over extensive documentation.
  - Informal communication is preferred over formal meetings and written documents.

The Process Models – Agile Process Model

- **Key Characteristics of Agile Methods**
- **Customer Involvement:**
  - Customers are actively involved in the development process.
  - They provide feedback and prioritize requirements.
- **Flexible Process:**
  - The process is adaptable to changing requirements and circumstances.
  - It emphasizes responding to change over following a rigid plan.

The Process Models – Agile Process Model

- **Key Characteristics of Agile Methods**
- **Continuous Testing:**
  - Testing is integrated into the development process.
  - Automated testing tools are used to ensure quality and reliability.
- **Tool Support:**
  - Tools are used to automate various aspects of the development process.
  - This includes version control, build automation, and testing tools.

The Process Models – Agile Process Model

Features	Plan-Driven Model	Agile Process Model
Process Model	Linear Flow	Iterative and Evolutionary
Requirements	Detailed Requirements Upfront	Evolving and Emerging
Documentation	Extensive	Minimal
Customer Involvement	Limited	High
Risk Management	Proactive	Reactive
Focus	Predictability and Control	Adaptability and Flexibility

The Process Models – Principles of Agile Methods

1. Customer Satisfaction	Early and continuous delivery of valuable Software
2. Embrace Change	Welcome changing requirements, even late in development.
3. Frequent Delivery	Deliver working software frequently, preferably in short cycles
4. Collaboration	Foster daily collaboration between businesspeople and developers.

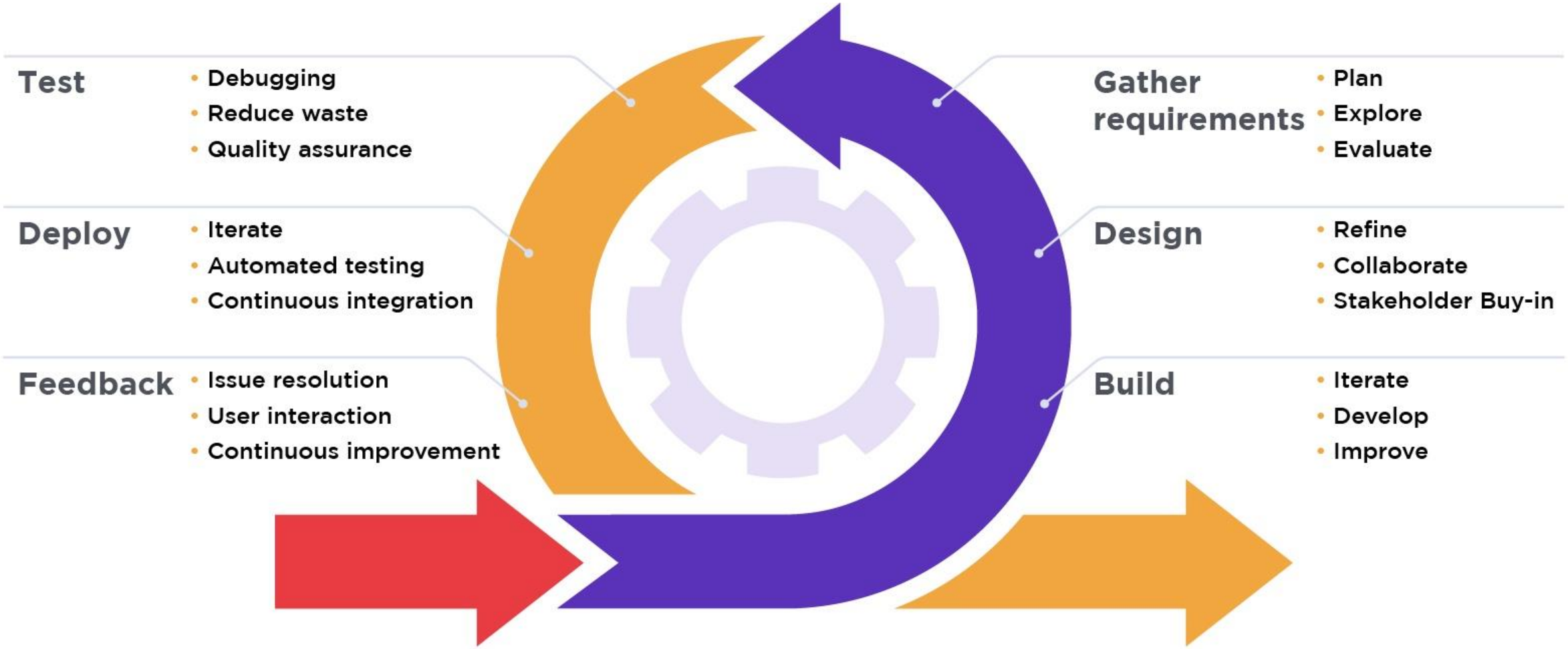
The Process Models – Principles of Agile Methods

5. Empowered Individuals	Build projects around motivated individuals and trust them
6. Face-to-Face Communication:	Prioritize face-to-face communication for effective information exchange.
7. Working Software	Prioritize working software as the primary measure of progress.
8. Sustainable Development	Promote sustainable development, avoiding excessive work hours.

The Process Models – Principles of Agile Methods

9. Technical Excellence	Continuously focus on technical excellence and good design.
10. Simplicity:	Embrace simplicity, avoiding unnecessary complexity.
11. Self-Organizing Teams:	Empower self-organizing teams to make decisions.
12. Continuous Reflection	Regularly reflect on team effectiveness and make adjustments.

The Process Models –Agile Method



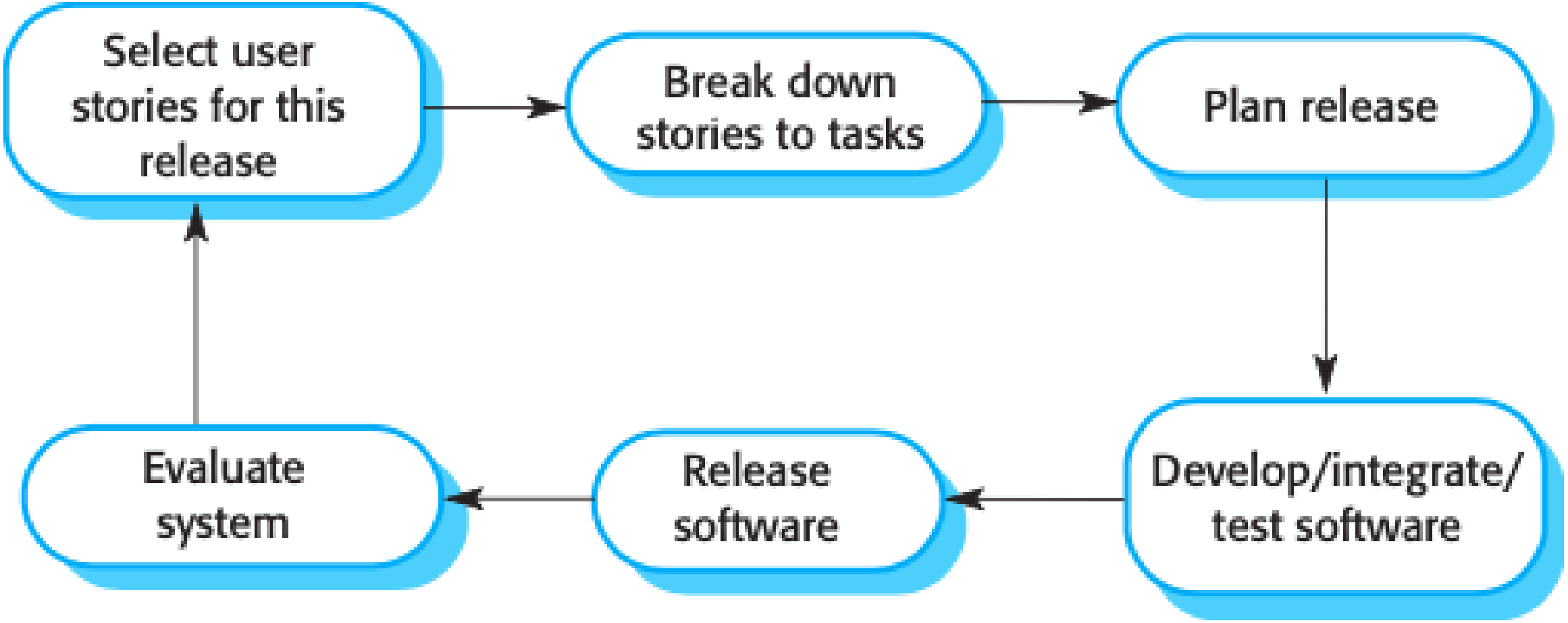
The Process Models –Agile Method – XP Principles

Principle or practice	Description
Collective ownership	The pairs of developers work on all areas of the system, so that no islands of expertise develop and all the developers take responsibility for all of the code. Anyone can change anything.
Continuous integration	As soon as the work on a task is complete, it is integrated into the whole system. After any such integration, all the unit tests in the system must pass.
Incremental planning	Requirements are recorded on "story cards," and the stories to be included in a release are determined by the time available and their relative priority. The developers break these stories into development "tasks." See Figures 3.5 and 3.6.
On-site customer	A representative of the end-user of the system (the Customer) should be available full time for the use of the XP team. In an extreme programming process, the customer is a member of the development team and is responsible for bringing system requirements to the team for implementation.
Pair programming	Developers work in pairs, checking each other's work and providing the support to always do a good job.

The Process Models –Agile Method – XP Principles

Principle or practice	Description
Refactoring	All developers are expected to refactor the code continuously as soon as potential code improvements are found. This keeps the code simple and maintainable.
Simple design	Enough design is carried out to meet the current requirements and no more.
Small releases	The minimal useful set of functionality that provides business value is developed first. Releases of the system are frequent and incrementally add functionality to the first release.
Sustainable pace	Large amounts of overtime are not considered acceptable, as the net effect is often to reduce code quality and medium-term productivity.
Test first development	An automated unit test framework is used to write tests for a new piece of functionality before that functionality itself is implemented.

The Process Models –Agile Method – Extreme Programming (XP)



The Process Models –Agile Method – Extreme Programming (XP)

- **Select User Stories for This Release:**
  - The development team, in collaboration with the customer, selects a set of user stories to be implemented in the current release.
  - User stories are high-level descriptions of the desired features or functionalities from the customer's perspective.

- **Break Down Stories to Tasks:**
  - Each selected user story is broken down into smaller, more manageable tasks.
  - These tasks are assigned to developers, who estimate the effort required to complete them.

The Process Models –Agile Method – Extreme Programming (XP)

- **Plan Release:**
  - Based on the selected user stories and estimated tasks, a release plan is created.
  - The plan outlines the timeline, milestones, and resource allocation for the release.

- **Develop, Integrate, and Test Software:**
  - Developers work in pairs (pair programming) to write code, with a focus on test-driven development (TDD).
  - Code is integrated into the main codebase frequently, and automated tests are run to ensure quality.

The Process Models –Agile Method – Extreme Programming (XP)

- **Release Software:**
  - Once the development and testing phases are complete, the software is released to the customer.
  - The customer evaluates the released software and provides feedback for the next iteration.

- **Evaluate System:**
  - The development team and the customer evaluate the released software to identify any issues or areas for improvement.
  - Feedback from the customer is used to refine future releases.

The Process Models –Agile Method – Extreme Programming (XP)

- Workflow Cycle:
  - Customer Defines User Stories: Customers describe what they need in simple, understandable terms.
  - Developers Estimate and Plan: Developers estimate the time and effort required for each story and plan iterations accordingly.
  - Build Iterations:
    - Write automated tests (TDD).
    - Pair program and write code to pass tests.
    - Refactor to improve the codebase.
    - Integrate and test frequently.

The Process Models –Agile Method – Extreme Programming (XP)

- **Deliver and Review:**
  - At the end of each iteration, deliver working software to the customer.
  - Gather feedback and adapt for the next iteration.

The Process Models –Agile Method – SCRUM

- Scrum is an Agile framework for managing and completing complex software development projects.
- It focuses on delivering high-value, high-quality software through collaboration, adaptability, and iterative development.
- Scrum divides work into sprints (time-boxed iterations, typically 1–4 weeks), where a specific set of goals or deliverables is planned, developed, tested, and delivered incrementally.

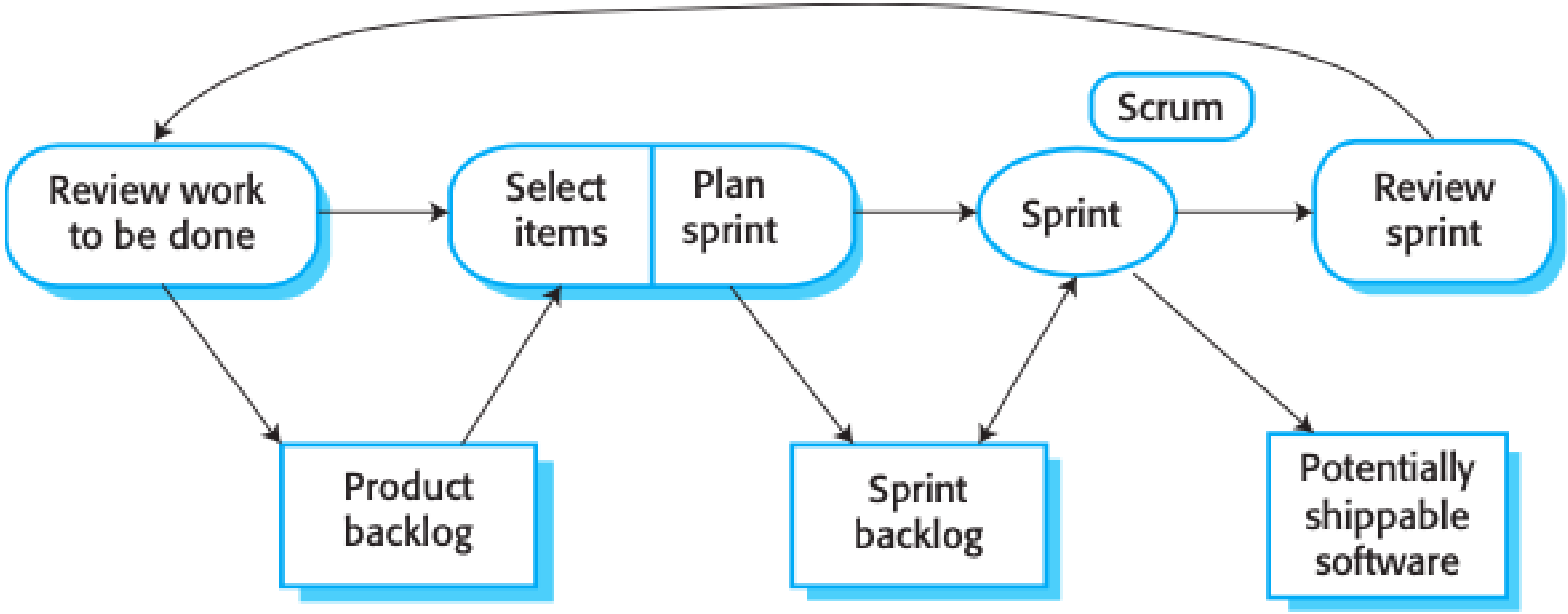
The Process Models –Agile Method – SCRUM Terminologies

Scrum term	Definition
Development team	A self-organizing group of software developers, which should be no more than seven people. They are responsible for developing the software and other essential project documents.
Potentially shippable product increment	The software increment that is delivered from a sprint. The idea is that this should be "potentially shippable," which means that it is in a finished state and no further work, such as testing, is needed to incorporate it into the final product. In practice, this is not always achievable.
Product backlog	This is a list of "to do" items that the Scrum team must tackle. They may be feature definitions for the software, software requirements, user stories, or descriptions of supplementary tasks that are needed, such as architecture definition or user documentation.
Product owner	An individual (or possibly a small group) whose job is to identify product features or requirements, prioritize these for development, and continuously review the product backlog to ensure that the project continues to meet critical business needs. The Product Owner can be a customer but might also be a product manager in a software company or other stakeholder representative.

The Process Models –Agile Method – SCRUM Terminologies

Scrum term	Definition
Scrum	A daily meeting of the Scrum team that reviews progress and prioritizes work to be done that day. Ideally, this should be a short face-to-face meeting that includes the whole team.
ScrumMaster	The ScrumMaster is responsible for ensuring that the Scrum process is followed and guides the team in the effective use of Scrum. He or she is responsible for interfacing with the rest of the company and for ensuring that the Scrum team is not diverted by outside interference. The Scrum developers are adamant that the ScrumMaster should not be thought of as a project manager. Others, however, may not always find it easy to see the difference.
Sprint	A development iteration. Sprints are usually 2 to 4 weeks long.
Velocity	An estimate of how much product backlog effort a team can cover in a single sprint. Understanding a team's velocity helps them estimate what can be covered in a sprint and provides a basis for measuring improving performance.

The Process Models –Agile Method – SCRUM Workflow



The Process Models –Agile Method – SCRUM Workflow

- **Product Backlog Creation:**
  - The Product Owner creates and maintains the product backlog.
  - Items are written as user stories (e.g., "As a user, I want to reset my password so that I can regain access to my account").
  - Each backlog item is prioritized based on value and urgency.

The Process Models –Agile Method – SCRUM Workflow

- **Sprint Planning:**
  - The team selects the highest-priority items from the product backlog to include in the sprint backlog.
  - The sprint goal is defined, and the team collaboratively plans how to deliver the selected backlog items.
  - Tasks for each backlog item are identified.

The Process Models –Agile Method – SCRUM Workflow

- **Sprint Execution:**
  - The development team works on the sprint backlog.
  - Progress is tracked, and daily stand-up meetings are held to ensure alignment and address blockers.
  - Continuous testing and integration are performed to ensure quality.

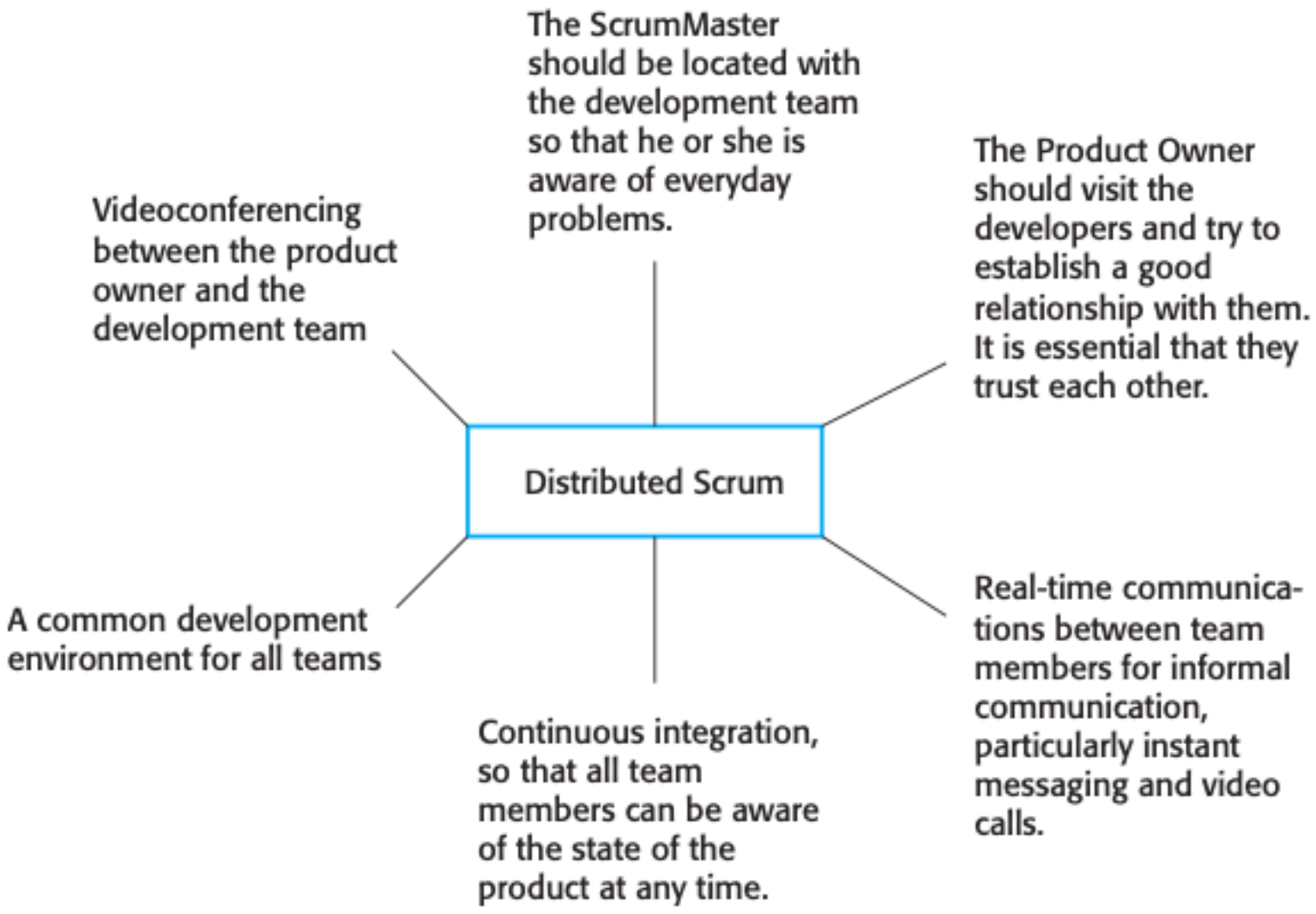
The Process Models –Agile Method – SCRUM Workflow

- **Sprint Review:**
  - At the end of the sprint, the team demonstrates the completed work (the increment) to stakeholders.
  - Stakeholders provide feedback, which helps refine the product backlog for future sprints.

The Process Models –Agile Method – SCRUM Workflow

- **Sprint Retrospective:**
  - The team reflects on the sprint process to identify successes and areas for improvement.
  - Adjustments are made to workflows or practices for the next sprint.
  - Repeat:
  - The next sprint begins, repeating the process with updated priorities and goals.

The Process Models –Agile Method – SCRUM Scaling



The Process Models –Reuse Oriented Software Engineering

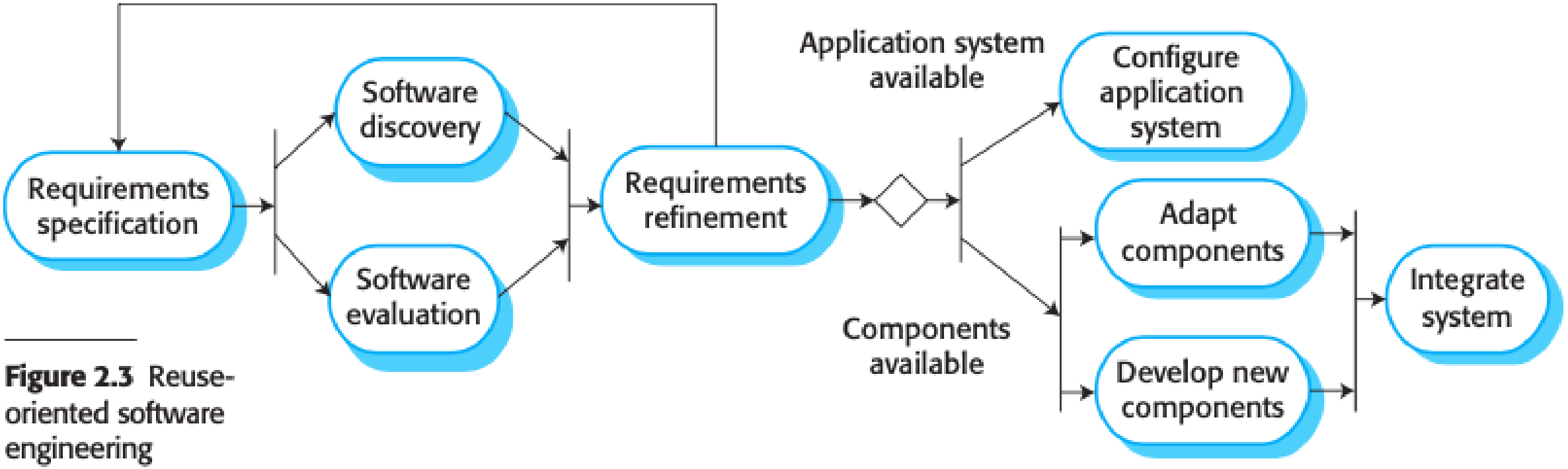


Figure 2.3 Reuse-oriented software engineering

The Process Models –Reuse Oriented Software Engineering

- **Requirements Specification:** Identify the initial system requirements.
- **Software Discovery and Evaluation:** Search for reusable components that meet the requirements.
- **Requirements Refinement:** Modify requirements to align with available components.
- **Application System Configuration:** Configure off-the-shelf systems if suitable.
- **Component Adaptation and Integration:** Modify and integrate components to build the system.